

Writz Protocol: Trustless Bitcoin Lending with Zero-Knowledge Position Privacy on Stellar

Sebastián Salazar Solano

Writz Protocol

Version 1.0 | July 2026

Abstract

We present Writz Protocol, a Bitcoin lending protocol on Stellar’s Soroban smart contract platform that combines SPV-verified native BTC collateral with zero-knowledge position privacy. Existing Bitcoin DeFi protocols force users to choose between custodial risk (delegating custody to a bridge operator) and public exposure (broadcasting position sizes to all network observers). Writz Protocol eliminates both constraints through the convergence of three production-ready technologies on Stellar: stateless Bitcoin SPV verification, Groth16 zero-knowledge proofs over the BN254 curve, and native USDC issuance.

The protocol makes three technical contributions. First, a stateless Bitcoin SPV client on Soroban that verifies P2WSH locking transactions cryptographically, without on-chain header storage or trusted relayers. Second, a Groth16 commitment scheme over BN254 that hides collateral amounts, loan balances, and liquidation thresholds from public observation while enabling sound on-chain enforcement of lending invariants via a ZK-friendly Poseidon Merkle tree. Third, a lending protocol (PrivateLend) with a two-slope interest rate model calibrated for Bitcoin collateral volatility, ZK-private liquidation proofs, and a multi-oracle price aggregator conforming to Stellar’s SEP-40 standard.

Four contracts are deployed on Soroban testnet with 114 tests passing; a complete deposit-borrow-repay ZK cycle has been verified on-chain against Soroban’s BN254 host functions introduced in Protocols 25 and 26. Bitcoin Signet end-to-end tests confirm P2WSH locking and co-signed release.

Keywords: Bitcoin, simplified payment verification, zero-knowledge proofs, Groth16, BN254, Poseidon, DeFi, lending protocol, Stellar, Soroban.

Disclaimer. This document is provided for informational purposes only. It does not constitute financial advice, an offering of securities, or a solicitation to purchase any token or financial instrument. The protocol is not yet available on mainnet. Current deployment information is maintained at <https://github.com/WritzProtocol/writz>.

Contents

1	Introduction	3
2	Background	4
2.1	Bitcoin Simplified Payment Verification	4
2.2	Groth16 Zero-Knowledge Proofs	5
2.3	Poseidon Hash Function	5
2.4	Stellar Soroban	6
3	Related Work	6
3.1	BTC Relay (Ethereum, 2016)	6
3.2	summa-tx/bitcoin-spv	6
3.3	Interlay/interBTC (Polkadot)	6
3.4	Aztec and Private DeFi	6
3.5	Stellar Private Payments	7
3.6	Comparative Summary	7
4	Protocol Design	7
4.1	System Overview	7
4.2	Bitcoin Collateral Layer	8
4.3	Stateless SPV Verification	9
4.4	ZK Privacy Layer	9
4.5	PrivateLend	12
5	Security Analysis	14
5.1	Threat Model	14
5.2	Bitcoin Custody Safety	14
5.3	Position Privacy	15
5.4	Nullifier Uniqueness and Double-Spend Resistance	15
5.5	Bitcoin-Side Risk Factors	16
5.6	Soroban-Side Risk Factors	16
6	Economic Model	16
6.1	Market Context	16
6.2	Revenue Streams	16
6.3	Fee Distribution	17
6.4	Governance Token	17
6.5	Conservative Revenue Projections	17
7	Implementation	17
7.1	Contract Suite	17
7.2	Compute Cost Analysis	18
7.3	Circuit Performance	18
8	Conclusion and Future Work	19
A	Testnet Deployment Records	20
A.1	Bitcoin Signet P2WSH End-to-End Test (2026-06-23)	21
A.2	On-Chain ZK Flow (Soroban Testnet, 2026-06-22)	21

1 Introduction

Bitcoin is the world’s largest crypto asset by market capitalization, with a supply that is both fixed and public. Yet participating in decentralized finance with native Bitcoin has historically required accepting one of two structural deficiencies.

The first deficiency is custodial risk. Protocols such as WBTC (custodied by BitGo), tBTC (secured by a threshold signature network), and Stacks sBTC (managed by a signer committee) require users to transfer economic control of their Bitcoin to an external party. Each of these designs introduces a trust assumption: the custodian must remain solvent, responsive, and honest. History has shown that this assumption fails. In 2023, BitGo’s announcement of a custody arrangement change triggered over one billion US dollars in WBTC redemptions within a single week.

The second deficiency is positional transparency. On every major DeFi lending protocol, including Aave, Compound, and Blend, every loan position is readable by any network observer. The collateral amount, outstanding debt, and liquidation price are publicly visible in real time. This creates a documented exploitation surface: automated bots monitor liquidation thresholds and trade against them, a phenomenon catalogued by Daian et al. as miner extractable value and sandwich attacks [1]. Institutional participants unwilling to expose their treasury positions to competitors have accordingly avoided the category.

Writz Protocol addresses both deficiencies simultaneously by combining three technologies that became production-ready on Stellar between September 2025 and January 2026:

1. **Bitcoin SPV on Soroban.** A Soroban smart contract verifies that a Bitcoin transaction was included in a confirmed block via a Merkle proof and a chain of block headers. The verification is stateless: no on-chain header chain is maintained, and no trusted relay is required. The cryptographic security derives from Bitcoin’s proof-of-work consensus.
2. **ZK proof verification on Stellar.** Stellar Protocol 25 (X-Ray, January 2026) embedded BN254 elliptic curve host functions and a Poseidon hash primitive into Soroban [2]. Protocol 26 (Yardstick, May 2026) added multi-scalar multiplication as a native host function, reducing Groth16 verification cost by approximately 40 percent [3]. Groth16 proofs generated by Circom circuits [4] are now verifiable on Stellar in a single transaction.
3. **Native USDC on Stellar.** Circle issues USDC natively on Stellar, not through a bridge. Borrowers receive the same USDC used by financial institutions for payments and settlement.

Contributions. This paper presents the Writz Protocol and makes the following contributions:

1. A stateless Bitcoin SPV client for Soroban, based on the `summa-tx/bitcoin-spv` Rust toolkit [5], that verifies P2WSH deposit transactions cryptographically without trusted intermediaries. The client requires 6 Bitcoin confirmations and operates within Soroban’s instruction budget at approximately 10 to 15 million instructions.
2. A zero-knowledge commitment scheme for lending positions using Circom, Groth16, and the Poseidon hash function [6]. Positions are encoded as tuples (c, d, s, n) (collateral, debt, secret, nonce) and committed via $C = \text{Poseidon}_4(c, d, s, n)$ in the BN254 scalar field [7]. Four circuit types enforce their respective invariants in zero-knowledge.
3. A complete lending protocol (PrivateLend) incorporating a kinked two-slope interest rate model calibrated for Bitcoin collateral volatility, a ZK-private liquidation mechanism, and a manipulation-resistant multi-oracle price aggregator.

4. Security propositions for custody safety, computational privacy, and double-spend resistance, with formal proof arguments and explicit trust assumptions.
5. Testnet implementation evidence: four contracts deployed on Soroban testnet, a complete on-chain ZK cycle, and a Bitcoin Signet P2WSH end-to-end test with confirmed broadcast (see Appendix A).

Paper organization. Section 2 introduces the cryptographic and protocol background. Section 3 surveys related work. Section 4 describes the full protocol design. Section 5 presents the security analysis. Section 6 covers the economic model. Section 7 documents the testnet implementation. Section 8 concludes with future work.

2 Background

Table 1 summarizes the notation used throughout this paper.

Table 1: Summary of notation.

Symbol	Description
$\mathbb{F}_p$	Scalar field of BN254, $ \mathbb{F}_p = p \approx 2^{254}$
$\mathbb{G}_1, \mathbb{G}_2$	Cyclic source groups of BN254
$\mathbb{G}_T$	Target group of the BN254 pairing
e	Bilinear pairing $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$
$\text{SHA256d}(\cdot)$	Double-SHA256: $\text{SHA256}(\text{SHA256}(\cdot))$
Poseidon_k	Poseidon hash with k field-element inputs, output in $\mathbb{F}_p$
$\text{Com}(P)$	Commitment of position P ; see Definition 4.2
$\text{Nul}(s, n)$	Nullifier derived from secret s and nonce n
c	Collateral amount in satoshis
d	Debt amount in USDC micro-units (stroops)
s	User position secret, $s \in \mathbb{F}_p$, drawn uniformly at random
n, n'	Position nonces in $\mathbb{F}_p$; n' freshly sampled at each transition
U	Pool utilization rate, $U = B/S$
B, S	Total borrowed and total supplied USDC
r_b, r_s	Annualized borrow and supply interest rates
p_{BTC}	BTC/USD oracle price in stroops per BTC
$\theta_{\min}$	Minimum collateral ratio in basis points (15,000 = 150%)
θ_{liq}	Liquidation threshold in basis points (12,000 = 120%)
R	Root of the on-chain Poseidon Merkle tree
π	Groth16 proof $(A, B, C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$

2.1 Bitcoin Simplified Payment Verification

Nakamoto’s original paper [8] introduced Simplified Payment Verification (SPV) in Section 8 as a method for verifying payment inclusion without running a full node. An SPV proof consists of two components: a sequence of Bitcoin block headers establishing sufficient proof-of-work, and a Merkle inclusion proof demonstrating that a specific transaction is committed within one of those blocks.

A Bitcoin block header is an 80-byte structure encoding version, previous block hash, Merkle root of transactions, timestamp, difficulty target, and nonce. The block hash is $\text{SHA256d}(\text{header})$, where

SHA256d denotes double-SHA256. Proof-of-work validity requires $\text{SHA256d}(\text{header}) < \text{target}(\text{bits})$.

A Merkle proof for transaction tx in a block with transactions $\{t_1, \dots, t_n\}$ is a sequence of sibling hashes $\{(h_i^{\text{sib}}, b_i)\}_{i=0}^{\lceil \log_2 n \rceil - 1}$ such that iterating from $h_0 = \text{SHA256d}(\text{txid})$ via:

$$h_{i+1} = \begin{cases} \text{SHA256d}(h_i \| h_i^{\text{sib}}) & \text{if } b_i = 0, \\ \text{SHA256d}(h_i^{\text{sib}} \| h_i) & \text{if } b_i = 1, \end{cases}$$

yields the block's Merkle root. Verification requires $O(\log_2 n)$ hash evaluations.

Stateless SPV. BTC Relay [9] maintained a full on-chain header chain in an Ethereum smart contract, creating a persistent liveness dependency on relayers. Stateless SPV, pioneered by summatx [5], avoids on-chain header storage: the caller provides the relevant block headers at call time, and the contract verifies them inline. Writz Protocol adopts the stateless approach.

P2WSH scripts. Pay-to-Witness-Script-Hash (P2WSH) locks funds to the SHA256 hash of a redeem script, with spending data carried in the transaction witness. Witness data is discounted at a factor of four in Bitcoin's weight calculation, making P2WSH economically efficient for multi-condition scripts.

2.2 Groth16 Zero-Knowledge Proofs

A succinct non-interactive argument of knowledge (SNARK) [10] is a protocol between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ for an NP relation $\mathcal{R}$. Given a public statement x , $\mathcal{P}$ convinces $\mathcal{V}$ that it knows a witness w such that $(x, w) \in \mathcal{R}$, without revealing w .

Groth16 [11] is a pairing-based SNARK with the following properties relevant to Writz Protocol:

- **Constant proof size.** A proof $\pi = (A, B, C)$ comprises two $\mathbb{G}_1$ elements and one $\mathbb{G}_2$ element, totaling 192 bytes on BN254 [7], regardless of circuit size.
- **Constant verification cost.** Verification reduces to a single bilinear pairing check. Given verification key $\text{vk} = (\alpha, \beta, \gamma, \delta, \{\text{IC}_i\}_{i=0}^m)$ and public inputs $\mathbf{x} = (x_1, \dots, x_m)$ with $x_0 = 1$, the verifier accepts if and only if:

$$e(-A, B) \cdot e(\alpha, \beta) \cdot e\left(\sum_{i=0}^m x_i \cdot \text{IC}_i, \gamma\right) \cdot e(C, \delta) = 1_{\mathbb{G}_T}. \quad (1)$$

- **Trusted setup requirement.** Groth16 requires a circuit-specific trusted setup. Security holds if at least one participant in the multi-party computation honestly destroys their randomness.

BN254 curve. BN254 (alt_bn128) is a pairing-friendly elliptic curve [7] with 254-bit prime-order scalar field $\mathbb{F}_p$. Stellar Protocol 25 added native host functions `bn254_add`, `bn254_scalar_mul`, `bn254_pairing`. Protocol 26 added `bn254_msm` for multi-scalar multiplication [2,3]. These execute at native speed outside the Wasm instruction budget, making Groth16 verification on Soroban both feasible and economically acceptable.

2.3 Poseidon Hash Function

SHA-256 and Keccak-256 are expensive inside ZK circuits because their bitwise operations require large R1CS constraint counts. Poseidon [6] is an algebraic hash function over $\mathbb{F}_p$ using a substitution-permutation network with MDS matrix and round constants. Its R1CS constraint count is 10 to

100 times lower than SHA-256 for equivalent security. Writz Protocol uses $\text{Poseidon}_4 : \mathbb{F}_p^4 \rightarrow \mathbb{F}_p$ for all position commitments and $\text{Poseidon}_2 : \mathbb{F}_p^2 \rightarrow \mathbb{F}_p$ for nullifiers. Stellar Protocol 25 CAP-0075 added Poseidon and Poseidon2 as native Soroban host functions.

2.4 Stellar Soroban

Soroban is Stellar’s WebAssembly-based smart contract execution environment. Contracts are metered across six resource dimensions: CPU instructions, ledger entry reads and writes, read and write bytes, and transaction size. The per-transaction instruction budget is approximately 100 million.

Protocol 23 (Whisk, September 2025) introduced parallel transaction execution. Protocol 25 (X-Ray, January 2026) added BN254 and Poseidon host functions [2]. Protocol 26 (Yardstick, May 2026) added nine BN254 arithmetic host functions, most significantly `bn254_msm`, and introduced checked 256-bit integer arithmetic (CAP-0082): overflows abort transactions rather than wrapping silently [3].

3 Related Work

3.1 BTC Relay (Ethereum, 2016)

BTC Relay [9] was the first Bitcoin SPV client on a general-purpose smart contract platform. It maintained an on-chain chain of Bitcoin block headers in an Ethereum smart contract. The project was abandoned due to two structural failures: relayers stopped submitting headers when fees declined, causing the on-chain chain to stall; and storing every Bitcoin block header on Ethereum was economically unviable. Writz Protocol avoids both problems through stateless SPV: no headers are stored on-chain, and the SPV proof is submitted at transaction time by the user.

3.2 summa-tx/bitcoin-spv

The `summa-tx bitcoin-spv` library [5] is a multi-chain stateless SPV toolkit with implementations in Rust, Solidity, Go, JavaScript, and Python. Its Rust implementation provides four functions: `validateHeader`, `validateHeaderChain`, `prove` (Merkle inclusion), and `extractTxOutputValue`. This library is the direct precursor to Writz Protocol’s `bitcoin-spv` Soroban contract.

3.3 Interlay/interBTC (Polkadot)

Interlay [12] is the most complete prior implementation of trustless Bitcoin DeFi on a non-Bitcoin chain. It combines an on-chain Bitcoin header chain (BTC-Relay pallet in Substrate/Rust) with overcollateralized vault operators who custody the BTC; if a vault steals funds, its collateral is automatically slashed and redistributed. The interBTC technical specification [12], which builds on the XCLAIM framework [13], is a direct reference for Writz Protocol’s SPV design. Writz Protocol differs from interBTC in two ways: it uses stateless rather than stateful SPV, and it provides ZK privacy for position amounts, which interBTC does not.

3.4 Aztec and Private DeFi

Aztec Network implements private DeFi on Ethereum using the PLONK proving system and a UTXO-based private state model. It is the closest functional analogue to Writz Protocol’s privacy layer. Key differences: Aztec targets Ethereum and uses PLONK (with a universal trusted setup);

it does not natively handle Bitcoin collateral; and its compliance model differs from Stellar’s Association Set Provider (ASP) framework. Tornado Cash, sanctioned by the US Office of Foreign Assets Control in 2022, hid the existence of transactions. Writz Protocol transactions are fully visible on-chain; only position amounts are kept private through ZK proofs. This distinction is material for regulatory analysis.

3.5 Stellar Private Payments

Nethermind’s Stellar Private Payments [14] is the direct open-source reference for Writz Protocol’s ZK layer. It implements a private asset pool on Soroban using Circom, Groth16, BN254, and browser-side WASM proof generation. Commitments are formed as $\text{Poseidon}_2(a, s)$ for asset amount a and secret s ; a Poseidon Merkle tree tracks active commitments; nullifiers prevent double-spending. Writz Protocol extends this model to the lending context by generalizing the commitment to encode both collateral and debt via $\text{Poseidon}_4(c, d, s, n)$ and by adding state-transition circuits that enforce collateral ratio invariants inside the proof.

3.6 Comparative Summary

Table 2 compares Writz Protocol to the most relevant prior systems across the dimensions that are central to its design goals.

Table 2: Comparison of Bitcoin DeFi and private lending protocols.

Protocol	Chain	Trustless BTC	ZK Privacy	Native USDC	Proof System	Proof
BTC Relay	Ethereum	Partial	No	No	None	N/A
interBTC	Polkadot	Yes (vaults)	No	No	None	N/A
Aztec	Ethereum	No	Yes	No	PLONK	~1 KB
Stellar PP	Stellar	No	Yes	No	Groth16	192 B
Writz Protocol	Stellar	Yes (SPV)	Yes	Yes	Groth16	192 B

Stellar PP = Stellar Private Payments (Nethermind). “Trustless BTC” means no human custodian is required for BTC security.

4 Protocol Design

4.1 System Overview

Writz Protocol operates across two independent blockchains with distinct roles. Bitcoin is the custody layer: BTC is locked in a script on the Bitcoin network and is never transferred to another chain. Stellar is the execution layer: loan logic, ZK verification, and USDC flows all execute on Soroban.

Figure 1 illustrates the two-chain architecture. The protocol has four layers: (1) the Bitcoin Network, where the user’s BTC wallet locks funds into a P2WSH address; (2) the Backend Services, comprising a stateless SPV Relayer that assembles proof bundles and a ZK Prover running in the user’s browser; (3) the Soroban Contracts, four contracts handling SPV verification, Groth16 verification, Poseidon Merkle tree management, and lending logic; and (4) the Browser, where private position data is generated and stored locally and never transmitted off-device.

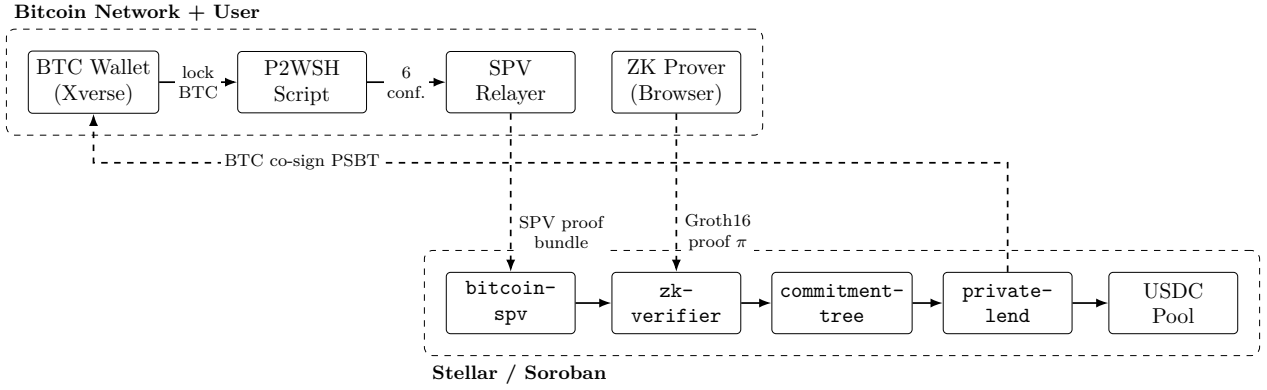


Figure 1: Two-chain architecture of Writz Protocol. Solid arrows denote on-chain transactions; dashed arrows denote cross-layer message passing. The BTC co-sign PSBT travels from the Soroban contract back to the user’s Bitcoin wallet for broadcast upon loan repayment.

4.2 Bitcoin Collateral Layer

4.2.1 P2WSH Locking Script

When a user initiates a deposit, the Writz Protocol frontend derives a unique P2WSH address from three parameters: the protocol’s co-signing public key $\mathbf{pk}_p$, the user’s Bitcoin public key $\mathbf{pk}_u$, and an absolute timelock value T . The address is computed as:

$$\text{script_hash} = \text{SHA256}(\text{redeemScript}), \quad \text{address} = \text{bech32}(\text{bc}, [\text{0x00}, \text{script_hash}]).$$

The redeem script encodes two spending conditions:

```
OP_IF
  <protocol_pubkey> OP_CHECKSIGVERIFY
  <user_pubkey>     OP_CHECKSIG
OP_ELSE
  <cltv_value> OP_CHECKLOCKTIMEVERIFY OP_DROP
  <user_pubkey> OP_CHECKSIG
OP_ENDIF
```

Condition A (cooperative release). The user repays their USDC loan. The PrivateLend Soroban contract emits a repayment event, the Writz Protocol backend co-signs a partially signed Bitcoin transaction (PSBT) spending Condition A, and the user adds their signature and broadcasts. The protocol never holds unilateral control: both signatures are required.

Condition B (unilateral recovery). If the protocol becomes unavailable, the user may spend the output after block height T using only their own key. The timelock is set to:

$$T = h_{\text{deposit}} + 4,320 + 1,008 \approx h_{\text{deposit}} + 37 \text{ days},$$

where $4,320 \approx 30 \times 24 \times 6$ blocks correspond to a 30-day loan and $1,008 \approx 7 \times 24 \times 6$ blocks to a 7-day safety buffer. A user who deposited at block height h_{deposit} may recover their BTC without any protocol involvement after approximately 37 days.

On-chain footprint. The P2WSH output is 34 bytes. A Condition A spend witness is approximately 257 bytes (user signature 72 bytes, protocol signature 71 bytes, selector byte 1 byte, redeem

script 114 bytes). Bitcoin’s 4x weight discount on witness bytes yields an effective weight of 149 vbytes, confirmed by the Signet broadcast documented in Appendix A.

Phase 2 upgrade path. P2TR (Taproot) with MuSig2 key aggregation would make cooperative releases indistinguishable from ordinary single-key Bitcoin payments on-chain, eliminating Bitcoin-side evidence of the locking arrangement. This upgrade is deferred to Phase 2 pending broader hardware wallet support for interactive MuSig2 signing.

4.2.2 Protocol Co-Signing Key

Phase 1 stores the protocol co-signing private key in a cloud Hardware Security Module (HSM). Key compromise under this model cannot steal BTC (Condition A requires the user’s signature), but could prevent cooperative releases, forcing users to the timelock path. Phase 2 replaces the single key with a threshold multi-party computation scheme (GG20/CGGMP21), distributing signing authority across independent operators. Stellar Protocol 27 authentication delegation further enables distributing co-signing authority at the Stellar level via delegated Soroban accounts.

4.3 Stateless SPV Verification

Algorithm 1 describes the core verification logic of the `bitcoin-spv` Soroban contract. The function is called `cross-contract` by the `commitment-tree` contract at deposit time.

The contract also exposes `EXTRACTOUTPUT`, which parses the raw Bitcoin transaction to verify that the correct satoshi amount was sent to the expected P2WSH address. The combination of these two functions provides a complete, trustless deposit confirmation.

Confirmation policy. Standard deposits require $m = 6$. A 6-block reorganization has not occurred in Bitcoin’s history, making this a negligible-risk threshold. A fast-lane option with $m = 3$ is available for deposits below a configurable cap.

Compute budget. The SPV verification requires approximately 10 to 15 million Soroban instructions: $O(\log_2 n)$ SHA256d evaluations for the Merkle proof (13 levels for 4,000 transactions) plus k header chain checks. The full SPV verification fits comfortably alongside Groth16 verification within the 100 million instruction budget.

4.4 ZK Privacy Layer

4.4.1 Commitment Scheme

Definition 4.1 (Position). *A position is a tuple $P = (c, d, s, n)$ where: $c \in \mathbb{Z}^+$ is the collateral in satoshis; $d \in \mathbb{Z}^+$ is the outstanding debt in USDC stroops; $s \in \mathbb{F}_p$ is a 253-bit secret drawn uniformly at random by the user; and $n \in \mathbb{F}_p$ is a 253-bit nonce drawn uniformly at random per state transition.*

Definition 4.2 (Commitment and Nullifier). *For a position $P = (c, d, s, n)$:*

$$C = \text{Com}(P) \triangleq \text{Poseidon}_4(c, d, s, n) \in \mathbb{F}_p, \quad (2)$$

$$N = \text{Nul}(s, n) \triangleq \text{Poseidon}_2(s, n) \in \mathbb{F}_p. \quad (3)$$

The commitment C is stored on-chain. The nullifier N is consumed on-chain when the state transitions to a new position $P' = (c, d', s, n')$ with freshly sampled n' .

Algorithm 1 VerifyTransaction

Require: $H = [h_0, \dots, h_k]$ (block headers, 80 bytes each); proof (Merkle sibling hashes); tx_index; raw_tx; m (minimum confirmations)

Ensure: (txid, block_hash) or **abort**

```

1: for  $i \leftarrow 0$  to  $k - 1$  do
2:   if  $\text{SHA256d}(h_i) \neq h_{i+1}.\text{prev\_block\_hash}$  then
3:     abort (chain broken)
4:   end if
5:   if  $\text{SHA256d}(h_i) \geq \text{target}(h_i.\text{bits})$  then
6:     abort (PoW invalid)
7:   end if
8: end for
9: if  $k + 1 < m$  then
10:  abort (insufficient confirmations)
11: end if
12: txid  $\leftarrow \text{SHA256d}(\text{raw\_tx})$ 
13:  $R \leftarrow \text{txid}$ 
14: for each  $(h_i^{\text{sib}}, b_i)$  in proof do
15:   if  $b_i = 0$  then
16:      $R \leftarrow \text{SHA256d}(R \| h_i^{\text{sib}})$ 
17:   else
18:      $R \leftarrow \text{SHA256d}(h_i^{\text{sib}} \| R)$ 
19:   end if
20: end for
21: if  $R \neq h_0.\text{merkle\_root}$  then
22:   abort (Merkle mismatch)
23: end if
24: return (txid,  $\text{SHA256d}(h_0)$ )

```

Commitments are organized in an on-chain Poseidon Merkle tree of depth 20, supporting up to $2^{20} = 1,048,576$ simultaneous positions. The empty tree root is the canonical depth-20 Poseidon zero-value:

$$R_0 = 0x2134e76ac5d21aab186c2be1dd8f84ee880a1e46eaf712f9d371b6df22191f3e.$$

Spent nullifiers are stored as individual 32-byte persistent ledger entries keyed by their value, preventing unbounded collection growth within instance storage.

4.4.2 Circuit Designs

All circuits are written in Circom [4], compiled to Rank-1 Constraint Systems (R1CS), and verified using Groth16 over BN254 via Soroban’s Protocol 25 and 26 host functions. The **zk-verifier** contract implements equation (1) using **bn254_msm** for the input-contribution sum and **bn254_pairing_check** for the final equation. The verification key IC array lengths (public input count plus one) are: Deposit = 6; BorrowRepay = 9; Liquidation = 6; ZeroDebt = 2. Constraint counts below are non-linear R1CS constraints as reported by the Circom compiler on the current circuit sources.

Deposit Circuit (596 constraints). Proves that a valid commitment has been formed for a BTC deposit without revealing the deposited amount:

- Private inputs: c, s, n
- Public inputs: $\text{txid_lo}, \text{txid_hi}, c_{\min}$
- Public outputs: $C = \text{Poseidon}_4(c, 0, s, n)$, $N = \text{Poseidon}_2(s, n)$

Enforced constraints: (i) $c \geq c_{\min}$ via a 52-bit greater-than comparator; (ii) d is initialized to zero; (iii) the txid binding inputs link the circuit to the specific on-chain Bitcoin transaction, preventing commitment replay.

Borrow/Repay Circuit (11,296 constraints). Proves that a state transition from $P = (c, d, s, n)$ to $P' = (c, d', s, n')$ is valid without revealing amounts:

- Private inputs: $c, d, s, n, n', \text{path}[20], \text{indices}[20]$
- Public inputs: $R_{\text{old}}, \Delta_d, \text{is_borrow}, p_{\text{BTC}}, \theta_{\min}$
- Public outputs: $R_{\text{new}}, N_{\text{old}} = \text{Poseidon}_2(s, n)$, $C_{\text{new}} = \text{Poseidon}_4(c, d', s, n')$

Enforced constraints: (i) $\text{Poseidon}_4(c, d, s, n)$ is a leaf in the tree with root R_{old} at the given path; (ii) $d' = d + \Delta_d$ with $d' \geq 0$ (120-bit range check), and range checks on the collateral and price inputs; (iii) is_borrow is boolean and consistent with the direction of the debt change (a borrow cannot decrease debt and a repayment cannot increase it); (iv) for borrows ($\text{is_borrow} = 1$), the collateral ratio invariant:

$$\frac{c \cdot p_{\text{BTC}}}{10^8} \cdot 10,000 \geq d' \cdot \theta_{\min};$$

(v) $\text{Poseidon}_4(c, d', s, n')$ replaces the old leaf, yielding R_{new} via the MERKLETREEUPDATER template.

Constraint breakdown: the cost is dominated by the two depth-20 Merkle operations (membership of the old leaf and update to the new root, approximately 5,200 constraints each), followed by the two Poseidon_4 commitment hashes (approximately 320 each) and the ratio, direction, and range checks.

Liquidation Circuit (5,710 constraints). Proves that a specific position is undercollateralized, enabling an authorized liquidator to claim the collateral:

- Private inputs: $c, d, s, n, \text{path}[20], \text{indices}[20]$
- Public inputs: $R, p_{\text{BTC}}, \theta_{\text{liq}}$
- Public outputs: $N = \text{Poseidon}_2(s, n)$, $d_{\text{repay}} = d$

Enforced constraints: (i) $\text{Poseidon}_4(c, d, s, n)$ is a leaf in the tree with root R ; (ii) the position is undercollateralized:

$$\frac{c \cdot p_{\text{BTC}}}{10^8} \cdot 10,000 < d \cdot \theta_{\text{liq}}.$$

Remark 4.1. *The liquidation circuit reveals $d_{\text{repay}} = d$ as a public output so the liquidator knows the USDC repayment amount. This partially breaks position privacy for liquidated positions. Phase 2 will evaluate encrypted-debt commitments or trusted-enclave keeper designs that preserve privacy through the liquidation event.*

Zero-Debt Release Circuit (5,220 constraints). Proves that a fully repaid position exists in the Merkle tree, providing the on-chain guarantee required to trigger the protocol co-signature for BTC release:

- Private inputs: $c, s, n, \text{path}[20], \text{indices}[20]$

- Public input: R
- Constraint: $\text{Poseidon}_4(c, 0, s, n)$ is a leaf in the tree with root R .

The debt field is hardcoded to zero in the constraint; a proof with any non-zero debt is impossible to generate for this circuit.

4.4.3 Trusted Setup Ceremony

Groth16 requires a per-circuit trusted setup proceeding in two phases.

Phase 1 (Powers of Tau). Writz Protocol reuses the Hermez Powers of Tau ceremony, a publicly audited multi-party computation with 90 independent participants that produced universal SRS parameters. This phase need not be repeated.

Phase 2 (circuit-specific). Each of the four circuits requires a separate MPC ceremony to produce the proving key pk and verification key vk . A minimum of 5 independent participants is required; security holds if at least one participant honestly destroys their randomness. The ceremony produces a publicly auditable transcript. This is a one-time pre-mainnet requirement. Current testnet deployments use development keys from the `pot15` powers-of-tau file, which must be replaced by ceremony keys before any mainnet deployment.

4.5 PrivateLend

4.5.1 Interest Rate Model

PrivateLend uses a two-slope (kinked) interest rate model. Let S denote total USDC supplied and B total USDC borrowed; the utilization rate is $U = B/S$. The borrow rate r_b and supply rate r_s are:

$$r_b(U) = \begin{cases} r_0 + \frac{U}{U^*} \cdot s_1 & \text{if } U \leq U^*, \\ r_0 + s_1 + \frac{U - U^*}{1 - U^*} \cdot s_2 & \text{if } U > U^*, \end{cases} \quad (4)$$

$$r_s(U) = r_b(U) \cdot U \cdot (1 - f). \quad (5)$$

Table 3: Interest rate model parameters.

Parameter	Symbol	Writz Protocol	Blend	Aave v3 (WBTC)
Base rate	r_0	0%	0%	0%
Optimal util.	U^*	75%	80%	45%
Slope 1	s_1	8%	8%	7%
Slope 2	s_2	200%	150%	300%
Protocol fee	f	15%	10%	10%

The choice of $U^* = 75\%$ (versus 80% for Blend) reflects BTC collateral volatility: a 40% BTC price decline can trigger simultaneous liquidations across many positions. Reserving 25% of the pool as free liquidity ensures USDC is available for liquidators during an acute drawdown. The protocol fee of 15% covers the additional on-chain cost of ZK proof verification per transaction.

Table 4: Borrow and supply rates at selected utilization levels.

Utilization U	Borrow rate r_b	Supply rate r_s
25%	2.67%	0.57%
50%	5.33%	2.27%
75%	8.00%	5.10%
80%	48.0%	32.6%
90%	128%	97.9%
95%	168%	134.7%

Interest accrues continuously per Stellar ledger close (approximately 5 seconds), at a per-ledger rate of $r_b/(365 \times 24 \times 720)$, using compound interest consistent with Aave and Blend.

4.5.2 Liquidation Mechanism

A position $P = (c, d, s, n)$ is liquidatable when its collateral ratio falls below $\theta_{\text{liq}} = 120\%$:

$$\frac{c \cdot p_{\text{BTC}}}{10^8 \cdot d} < 1.20.$$

Worked example. A user deposits 1 BTC at $p_{\text{BTC}} = \$100,000$ and borrows \$60,000 USDC (collateral ratio = 167%). The position becomes liquidatable at $p_{\text{BTC}} = \$72,000$ (ratio = 119%). With accrued interest bringing the debt to \$60,500: the liquidator pays \$60,500 USDC; receives \$66,550 in BTC (10% bonus); the protocol retains \$1,210 in BTC (2% fee); the borrower recovers the \$4,240 residual. Table 5 lists the liquidation parameters.

Table 5: Liquidation parameters.

Parameter	Value	Notes
Minimum collateral ratio	150%	Max LTV = 66.7%
Liquidation threshold	120%	Trigger for liquidation
Collateral buffer	30 pp	Margin for BTC volatility
Liquidation bonus	10%	Liquidator’s BTC discount
Protocol liquidation fee	2%	Routes to insurance fund
Minimum liquidation size	10%	Of outstanding debt per call

Keeper architecture (Phase 1). Because position amounts are hidden, automated bots cannot scan the chain for undercollateralized positions. Phase 1 employs a privileged keeper: an off-chain service that holds position preimages received via an encrypted channel at deposit time, monitors the BTC/USD oracle, and generates liquidation ZK proofs when a position falls below the threshold. The keeper publishes the proof on-chain; any party may act as the USDC-paying liquidator once the proof is available. An emergency fallback in the contract allows open liquidation if no valid proof is submitted within a configured block window, preventing keeper downtime from leaving undercollateralized positions unaddressed.

4.5.3 Oracle Design

PrivateLend requires a manipulation-resistant BTC/USD price feed for collateral ratio calculations and liquidation triggers. The oracle module implements a multi-source median:

$$p_{\text{BTC}} = \text{median}(p_1, p_2, \dots, p_k),$$

where each p_i is drawn from an independent institutional price source. Phase 1 uses two sources: RedStone (primary, SEP-40 standard, live on Stellar mainnet since March 2026) and Pyth Network (secondary). The contract interface is defined against the SEP-40 oracle abstraction, allowing provider substitution without contract changes.

Additional safeguards: (i) a staleness threshold of 60 minutes (stale prices pause liquidations but not new borrows); (ii) a maximum per-source deviation of 5% from the median; (iii) a liquidation price floor of $\min(p_{\text{current}}, p_{t-5\text{min}})$ to prevent short-lived price spikes from triggering liquidations. Flash loan manipulation is structurally prevented by using off-chain institutional price feeds rather than on-chain DEX prices.

5 Security Analysis

5.1 Threat Model

We model an adversary $\mathcal{A}$ with the following capabilities: (i) $\mathcal{A}$ observes the complete public state of both the Bitcoin and Stellar blockchains; (ii) $\mathcal{A}$ may submit arbitrary transactions to either chain; (iii) $\mathcal{A}$ may compromise up to $k - 1$ of k Phase 2 MPC co-signers; (iv) $\mathcal{A}$ may corrupt at most one of the two oracle sources. We assume Bitcoin consensus security (no 51% attack), the soundness of Groth16 under the knowledge of exponent (KoE) assumption over BN254, and that Poseidon₄ and Poseidon₂ are pseudorandom functions over $\mathbb{F}_p$. Table 6 summarizes the trust assumptions and their violation consequences.

Table 6: Trust assumptions and violation consequences.

Component	Assumption	Consequence if violated
Bitcoin consensus	No 51% attack; script enforced	Catastrophic
Soroban contracts	Code executes as written	Critical
Groth16 circuits	Soundness holds (KoE)	Critical
Trusted setup	At least one participant honest	Critical
Protocol co-sign key	HSM not compromised (Phase 1)	High
Oracle price feeds	At most one source corrupted	High
User private keys	Not compromised by user	User’s risk

5.2 Bitcoin Custody Safety

Proposition 5.1 (Custody Safety). *For any P2WSH output committed to $\text{redeemScript}(\text{pk}_p, \text{pk}_u, T)$, any valid Bitcoin transaction spending that output must either: (a) include valid signatures from both sk_p and sk_u ; or (b) have block height $\geq T$ and include a valid signature from sk_u .*

Proof. Bitcoin Script execution is deterministic and independently enforced by every full node. The OP_IF branch calls OP_CHECKSIGVERIFY on pk_p , which terminates script execution with failure if the protocol signature is absent or invalid, followed by OP_CHECKSIG on pk_u . The OP_ELSE branch calls

OP_CHECKLOCKTIMEVERIFY with absolute value T , which fails if the transaction's lock time is below T or the block height is below T , followed by OP_CHECKSIG on pk_u . The redeem script contains no other spending paths, and the P2WSH commitment to the script hash ensures that no alternative script can be substituted. Therefore, every valid spending transaction satisfies either (a) or (b). $\square$

Corollary 5.1. *Protocol shutdown does not result in permanent BTC loss. A user with an active loan recovers their BTC after block height T using only their own Bitcoin wallet, retaining the borrowed USDC.*

5.3 Position Privacy

Proposition 5.2 (Computational Privacy). *Let $C = \text{Poseidon}_4(c, d, s, n)$ be a commitment with $s, n \xleftarrow{\$} \mathbb{F}_p$. Given C and the complete public blockchain state (all commitments, nullifiers, oracle prices, and aggregate TVL), any probabilistic polynomial-time adversary $\mathcal{A}$ has negligible advantage in distinguishing any two pairs (c_0, d_0) and (c_1, d_1) as the preimage of C , under the assumption that Poseidon_4 is a pseudorandom function (PRF) over $\mathbb{F}_p$.*

Proof. Suppose for contradiction that $\mathcal{A}$ distinguishes (c_0, d_0) from (c_1, d_1) given C with non-negligible advantage ε . We construct a PRF adversary $\mathcal{B}$ against Poseidon_4 as follows. $\mathcal{B}$ receives an oracle $\mathcal{O}$ that is either Poseidon_4 or a truly random function $\mathcal{F} : \mathbb{F}_p^4 \rightarrow \mathbb{F}_p$. $\mathcal{B}$ samples $s, n \xleftarrow{\$} \mathbb{F}_p$, queries $C_0 = \mathcal{O}(c_0, d_0, s, n)$ and $C_1 = \mathcal{O}(c_1, d_1, s, n)$, and forwards one commitment to $\mathcal{A}$.

If $\mathcal{O} = \text{Poseidon}_4$, then $\mathcal{A}$ succeeds with advantage ε by hypothesis. If $\mathcal{O} = \mathcal{F}$, then C is uniformly distributed in $\mathbb{F}_p$ independently of (c, d) , so $\mathcal{A}$'s advantage is at most $1/p$. Therefore $\mathcal{B}$'s advantage in distinguishing Poseidon_4 from $\mathcal{F}$ is at least $\varepsilon - 1/p$. Since $p \approx 2^{254}$, the term $1/p$ is negligible. If ε were non-negligible, $\mathcal{B}$ would break the PRF security of Poseidon_4 , contradicting our assumption. $\square$

Remark 5.1. *Proposition 5.2 does not apply to liquidated positions: the liquidation circuit reveals $d_{\text{repay}} = d$ as a public output, as noted in Section 4.4.2.*

5.4 Nullifier Uniqueness and Double-Spend Resistance

Proposition 5.3 (No Double-Spend). *Once a nullifier $N = \text{Poseidon}_2(s, n)$ is recorded in the on-chain nullifier set, no valid borrow, repay, or liquidation proof can be accepted for any position that derives the same nullifier.*

Proof. Each Borrow/Repay and Liquidation circuit constrains the public output N_{old} to equal $\text{Poseidon}_2(s, n)$, where (s, n) are the private witnesses for the consumed commitment $\text{Poseidon}_4(c, d, s, n)$. Under the KoE assumption, the circuit's soundness guarantees that no prover can produce a valid proof with an N_{old} inconsistent with a legitimately known commitment preimage.

The `commitment-tree` contract checks that N is absent from the on-chain nullifier set before accepting any proof; if N is present, the transaction aborts. Each accepted state transition produces a fresh commitment $\text{Poseidon}_4(c, d', s, n')$ with a new nonce $n' \xleftarrow{\$} \mathbb{F}_p$. The probability that $n' = n$ is $1/p \approx 2^{-254}$, which is negligible, so the new nullifier $\text{Poseidon}_2(s, n')$ is distinct from $\text{Poseidon}_2(s, n)$ with overwhelming probability. Therefore a recorded nullifier can never be re-derived from a fresh state transition. $\square$

5.5 Bitcoin-Side Risk Factors

Chain reorganization. Six-block reorganizations have not occurred in Bitcoin’s history. Requiring $m = 6$ confirmations makes acceptance susceptible to reorgs with negligible probability.

Protocol key compromise (Phase 1). An adversary who obtains sk_p cannot unilaterally move BTC (Condition A requires sk_u). The adversary could refuse to co-sign repayment releases, forcing users to the timelock path. Phase 2 MPC with t -of- n threshold ECDSA eliminates this single-key dependency.

P2WSH script correctness. The redeem script is 114 bytes and follows a well-understood two-condition pattern. Formal verification of the script is targeted for the pre-mainnet audit scope.

5.6 Soroban-Side Risk Factors

ZK circuit soundness. A soundness bug would allow undercollateralized borrowing by generating a false proof. Mitigations: a pre-mainnet ZK circuit audit (Veridise is the target engagement); progressive TVL caps; an on-chain insurance fund seeded from protocol fees.

Oracle manipulation. Manipulating the median of two independent institutional feeds requires both sources to be simultaneously compromised. The 5% deviation filter and staleness threshold provide additional resilience. Flash loan manipulation is structurally prevented by off-chain price sourcing.

Arithmetic overflow. Protocol 26 CAP-0082 checked 256-bit arithmetic makes overflows explicit transaction aborts rather than silent wrapping, eliminating a classical DeFi exploit vector in financial calculations.

Unbounded storage growth. Commitments and nullifiers are stored as individual 32-byte persistent ledger entries keyed by their hash, not as elements of a growing collection in instance storage. Storage cost per user is bounded and predictable.

6 Economic Model

6.1 Market Context

The Bitcoin DeFi (BTCfi) sector grew from approximately \$304 million in total value locked (TVL) in January 2024 to over \$8.6 billion by mid-2025, a 28-fold increase in 18 months. This growth occurred despite the structural deficiencies described in Section 4: custodial bridge risk and public position exposure. Institutional capital, which requires privacy as a baseline condition, has largely remained outside the category because of these deficiencies.

As of July 2026, existing Bitcoin lending designs either hold the collateral with a custodian or publish every position, and none combines SPV-verified native BTC collateral with ZK-private positions.

6.2 Revenue Streams

Primary (lending spread). PrivateLend retains 15% of the interest paid by borrowers. At \$1 million TVL and 75% utilization:

$$\text{Annual spread revenue} = \$750,000 \times 8\% \times 0.15 \approx \$9,000/\text{year}.$$

Secondary (SPV API). Third-party Stellar protocols pay per verification call (\$0.10 to \$0.50) or via monthly subscription (\$500 to \$5,000).

Tertiary (Dark Swap, Phase 3). ZK-private BTC/USDC swaps charge 0.30% per swap volume.

Quaternary (ZK Proof of Reserve, Phase 3). Enterprise clients pay \$500 to \$10,000 per month for private, verifiable Bitcoin reserve attestations for auditor and regulator submission.

6.3 Fee Distribution

Protocol revenue →	30%	Insurance fund
	30%	WRTZ buyback and burn
	25%	Operations treasury
	15%	Ecosystem grants

The insurance fund accumulates until it reaches 10% of TVL, after which excess revenue redirects to the buyback-and-burn mechanism.

6.4 Governance Token

WRTZ is a fixed-supply governance token (100 million tokens, no inflation). Token utility includes: protocol parameter governance (interest rate curves, collateral ratios, fee splits), revenue capture via buyback-and-burn, USDC yield boost for stakers, and priority access to liquidation opportunities for staked keeper operators. The token launch criterion is \$5 million TVL sustained for 60 consecutive days with at least one completed external audit, targeting Q4 2027 to Q1 2028, following a period of proven mainnet operation after the Q2 to Q3 2027 launch.

6.5 Conservative Revenue Projections

Table 7: Conservative annual revenue projections.

Revenue stream	Year 1 (post-mainnet)	Year 2 (post-mainnet)
Lending spread	\$18,000	\$180,000
Dark Swap fees	\$18,000	\$180,000
SPV API	\$6,000	\$60,000
Proof of Reserve	\$60,000	\$300,000
Liquidation fees	\$12,000	\$60,000
Total	\$114,000	\$780,000
Assumed TVL	\$2M	\$20M

Years are counted from mainnet launch, targeted for Q2 to Q3 2027.

7 Implementation

7.1 Contract Suite

Four Soroban contracts form the complete on-chain implementation. Table 8 lists each contract by name, WASM size, and test count. Current testnet deployment addresses and WASM hashes are maintained in the project repository at <https://github.com/WritzProtocol/writz>; deployment

records are versioned alongside each contract release and should be consulted directly rather than from this document, which describes a point in time.

Table 8: Soroban contract suite (Soroban testnet, July 2026).

Contract	WASM Size	Tests
bitcoin-spv	5,225 B	28
zk-verifier	11,781 B	18
commitment-tree	26,580 B	18
private-lend	26,477 B	50
Total	70,063 B	114

7.2 Compute Cost Analysis

Table 9: Estimated Soroban instruction costs per operation (post-Protocol 26 host functions).

Operation	Instructions	Approx. Cost (XLM)
Groth16 ZK verification	20 to 25M	0.02 to 0.03
Bitcoin SPV (6 headers)	10 to 15M	0.01 to 0.02
Lending state update	1 to 3M	0.001 to 0.003
Full deposit (SPV + ZK + state)	31 to 43M	0.031 to 0.053

At a current XLM price of approximately \$0.40, a full deposit operation costs between \$0.012 and \$0.021. Comparable operations on Ethereum routinely cost \$5 to \$50 in gas fees.

Transaction payload sizes: six Bitcoin block headers ($6 \times 80 = 480$ bytes), Merkle proof (13 sibling hashes, $13 \times 32 = 416$ bytes), raw Bitcoin transaction (200 to 1,000 bytes), Groth16 proof (192 bytes fixed). Total transaction payload: approximately 1.5 to 2.5 KB, within Soroban’s transaction size limit.

7.3 Circuit Performance

Table 10 reports proof generation times measured using snarkjs compiled to WebAssembly, run under Node.js on a standard consumer laptop. Each time covers witness generation and proving as exercised by the circuit test suite; in-browser measurements are a pre-mainnet task.

Table 10: Proof generation times (snarkjs, WASM, Node.js).

Circuit	Constraints (non-linear)	Proof Time
Deposit	596	< 1 second
Borrow/Repay	11,296	1 to 3 seconds
Liquidation	5,710	1 to 2 seconds
Zero-Debt	5,220	1 to 2 seconds

These times are acceptable for DeFi operations, given that users already wait 10 to 60 minutes

for Bitcoin block confirmations. GPU-accelerated proving is available as an optional upgrade for institutional operators. Testnet transaction records are documented in Appendix A.

8 Conclusion and Future Work

We have presented Writz Protocol, a trustless, privacy-preserving Bitcoin lending protocol on Stellar’s Soroban platform. The three core contributions are: a stateless Bitcoin SPV client for Soroban that verifies P2WSH deposit transactions without trusted intermediaries or on-chain header storage; a Groth16 zero-knowledge commitment scheme over BN254 that hides collateral amounts, loan balances, and liquidation thresholds from public observation while enabling sound on-chain enforcement of lending invariants; and a lending protocol with a two-slope interest rate model calibrated for Bitcoin collateral volatility, ZK-private liquidations, and a manipulation-resistant multi-oracle price aggregator.

As of July 2026, four Soroban contracts are deployed on testnet with 114 passing tests. A complete six-transaction on-chain ZK cycle has been verified on-chain. A Bitcoin Signet P2WSH end-to-end test confirms the locking and co-signed release flow. Mainnet launch is targeted for Q2 to Q3 2027, beginning with a gated, TVL-capped release of PrivateLend.

Phase 2.

- Replace the HSM co-signing key with threshold MPC (GG20/CGGMP21) and leverage Stellar Protocol 27 authentication delegation to distribute signing authority across independent operators.
- Replace the privileged keeper with a decentralized keeper network using a staked-bond mechanism, where any operator can compete to submit liquidation proofs and earn the liquidation bonus.
- Upgrade Bitcoin-side locking from P2WSH to Taproot (P2TR) with MuSig2 key aggregation, making cooperative releases indistinguishable from ordinary single-key Bitcoin payments.

Phase 3.

- *Dark Swap*: ZK-private BTC/USDC atomic swaps using the SPV contract for deposit verification and a Groth16 circuit for amount privacy.
- *BTC Savings*: Automatic yield routing of borrowed USDC into the highest-yield pools on Stellar (Blend, AMMs).
- *ZK Proof of Reserve (B2B)*: Enterprise service for private, verifiable Bitcoin reserve attestations for auditor and regulatory submission.
- *BitVM integration (research track)*: BitVM enables Bitcoin to verify ZK proofs via an optimistic challenge-response mechanism, potentially enabling trustless bidirectional state verification between Bitcoin and Stellar. This direction is tracked as a long-term research item pending BitVM production readiness.

Post-quantum preparedness. Stellar’s quantum preparedness roadmap (Stage 2, 2027) will introduce opt-in post-quantum signer types (ML-DSA-44, ML-DSA-65) for existing accounts. Because Stellar separates account identity from signing keys, Writz Protocol protocol keys can be migrated to post-quantum signatures without redeploying contracts or modifying user positions.

References

- [1] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, “Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability,” in *Proceedings of the 2020 IEEE Symposium on Security and Privacy*. IEEE, 2020, pp. 910 to 927.
- [2] Stellar Development Foundation, “Announcing Stellar X-Ray, Protocol 25: Zero-Knowledge Proof Verification on Soroban,” <https://stellar.org/blog/developers/announcing-stellar-x-ray-protocol-25>, 2026, published January 2026. Accessed July 2026.
- [3] Stellar Development Foundation, “Yardstick: Stellar Protocol 26,” <https://stellar.org/blog/foundation-news/yardstick-stellar-protocol-26>, 2026, published May 2026. Accessed July 2026.
- [4] M. Bellés-Muñoz, B. Whitehat, J. Baylina, V. Daza, and J. L. Muñoz-Tapia, “Circom: A Robust and Scalable Language for Building Complex Zero-Knowledge Circuits,” *Cryptology ePrint Archive*, Report 2022/1877, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1877>
- [5] summa-tx, “bitcoin-spv: Tools for Working with Bitcoin SPV Proofs from Other Chains,” GitHub repository, <https://github.com/summa-tx/bitcoin-spv>, 2019, accessed July 2026.
- [6] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A New Hash Function for Zero-Knowledge Proof Systems,” in *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, 2021, pp. 519 to 535.
- [7] P. S. L. M. Barreto and M. Naehrig, “Pairing-Friendly Elliptic Curves of Prime Order,” in *Selected Areas in Cryptography, SAC 2005*, ser. Lecture Notes in Computer Science, vol. 3897. Springer, 2006, pp. 319 to 331.
- [8] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” <https://bitcoin.org/bitcoin.pdf>, 2008, accessed July 2026.
- [9] Ethereum Foundation, “BTC Relay: A Solidity Contract for Bitcoin SPV Proof Verification,” GitHub repository, <https://github.com/ethereum/btcrelay>, 2016, deprecated. Accessed July 2026.
- [10] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, “Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture,” in *Proceedings of the 23rd USENIX Security Symposium*. USENIX Association, 2014, pp. 781 to 796.
- [11] J. Groth, “On the Size of Pairing-Based Non-Interactive Arguments,” in *Advances in Cryptology, EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305 to 326.
- [12] Interlay, “interBTC Technical Specification,” <https://spec.interlay.io>, accessed July 2026.
- [13] A. Zamyatin, D. Harz, J. Lind, P. Panayiotou, A. Gervais, and W. J. Knottenbelt, “XCLAIM: Trustless, Interoperable, Cryptocurrency-Backed Assets,” in *Proceedings of the 2019 IEEE Symposium on Security and Privacy*. IEEE, 2019, pp. 193 to 210.
- [14] Nethermind, “Stellar Private Payments: Privacy-Preserving Asset Transfers on Soroban,” GitHub repository, <https://github.com/NethermindEth/stellar-private-payments>, 2026, accessed July 2026.

A Testnet Deployment Records

This appendix documents the on-chain evidence from testnet deployments, current as of July 2026. These records reference specific transaction hashes and addresses that are immutable on the Bitcoin Signet and Soroban testnet blockchains. Current mainnet deployment addresses, when available, will be published at <https://github.com/WritzProtocol/writz>.

A.1 Bitcoin Signet P2WSH End-to-End Test (2026-06-23)

A P2WSH address was derived from protocol key 031918f1cf7f... and user key 02bbcf244d0b... with CLTV value 700,000 blocks. The deposit and cooperative release flow was confirmed on Bitcoin Signet:

Table 11: Bitcoin Signet P2WSH test results.

Event	Value
Funding transaction (89,631 sat to P2WSH)	61deea44...
Release transaction (88,131 sat to user)	11932100...
Transaction size	347 bytes (149 vbytes)
Fee	1,500 sat (10.1 sat/vbyte)
Witness items	user sig (72 B), protocol sig (71 B), selector (1 B), redeemScript (114 B)

Verified: P2WSH address derived correctly; both keys signed the PSBT independently; the finalization step for Path A assembled the correct Condition A witness; transaction broadcast and accepted by Bitcoin Signet mempool.

A.2 On-Chain ZK Flow (Soroban Testnet, 2026-06-22)

A complete deposit-borrow-repay ZK cycle was executed in six sequential transactions on Soroban testnet, using XLM as a USDC substitute for testnet purposes.

Table 12: On-chain ZK flow verification on Soroban testnet.

Step	Transaction	Verified
Initialize	b6527209...	Pool initialized
Supply 500 XLM	d340e516...	Lender balance updated
Deposit (ZK proof)	ae3c8eeb...	Groth16 BN254 pairing; Merkle root updated
Insert commitment	32d263a5...	Poseidon root confirmed
Borrow 200 XLM (ZK)	046a1165...	150% ratio enforced; USDC transferred
Repay 200 XLM (ZK)	11c94e34...	Repay proof; pool accounting updated

The Poseidon Merkle root after the deposit step matched the expected value for a depth-20 tree with one non-zero leaf, confirming correct Poseidon hash evaluation via the BN254 host functions. All Groth16 proofs were verified against the `zk-verifier` contract using Protocol 25 and 26 host functions.